PRACTICAL SESSION

# Deploying a Live Project with Docker

On Oracle Linux 9: containers, Git, webhooks, GitHub Actions and troubleshooting



CONTENTS

# What we'll cover


01 Containers on Windows 0:03

02 Linux toolkit for Oracle Linux 9 0:12

03 AMS case study & practice project 0:20

04 Preparing the server 0:29

05 Deploy the sample step by step 0:36

06 Build without installing anything 0:50

07 Git, webhooks and GitHub Actions 0:56

08 Troubleshooting and keeping it running 1:10

09 Subdomains with image IDs 1:21

01

# Containers on Windows



Why Docker on Windows always runs a virtual machine

CONTAINERS ON WINDOWS

# A container needs a Linux kernel

- A container shares the kernel of the machine it runs on
- Linux images expect Linux system calls
- Windows has its own NT kernel, not Linux

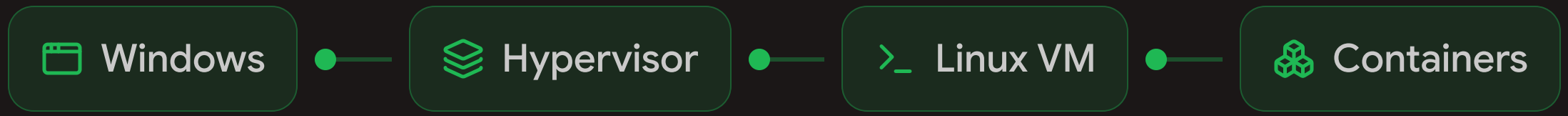


Windows containers exist, but almost every image on Docker Hub is Linux

CONTAINERS ON WINDOWS

# So Windows runs Linux underneath

Docker Desktop quietly starts a small Linux virtual machine.




Two ways to get that VM: Hyper-V or WSL 2

CONTAINERS ON WINDOWS

# What is a hypervisor?

- Software that runs virtual machines on real hardware
- **Type 1** runs on the hardware: Hyper-V, KVM, ESXi
- **Type 2** runs as an app: VirtualBox, VMware Workstation
- Needs CPU virtualisation, Intel VT-x or AMD-V, on in BIOS

CONTAINERS ON WINDOWS

# Hyper-V vs WSL 2

## Hyper-V

Windows Pro, Enterprise or Server

A full VM you manage

Docker's older backend

## WSL 2

Works on Windows Home too

A real Linux kernel in a light VM

Docker Desktop's default

CONTAINERS ON WINDOWS

# Why WSL 2 and not WSL 1?

- **WSL 1** translates Linux calls into Windows calls: no real kernel
- **WSL 2** runs Microsoft's real Linux kernel
- Docker needs namespaces and cgroups, so only WSL 2 works
- Keep projects inside the Linux filesystem for speed

CONTAINERS ON WINDOWS

# Trace it: is virtualisation on?

```
PS> systeminfo | findstr /i "hyper-v"
Hyper-V Requirements: A hypervisor has been detected.
PS> bcdedit /enum | findstr /i hypervisorlaunchtype
hypervisorlaunchtype    Auto
PS> Get-WindowsOptionalFeature -Online -FeatureName VirtualMachinePlatform
```

Task Manager, Performance, CPU: look for Virtualization: Enabled

CONTAINERS ON WINDOWS

# Trace it: WSL

```
PS> wsl --version                # WSL and kernel versions
PS> wsl --status                 # default distro and version
PS> wsl -l -v
  NAME                   STATE           VERSION
* Ubuntu                 Running         2
  docker-desktop         Running         2
PS> wsl --shutdown               # stop every WSL VM
```

CONTAINERS ON WINDOWS

# Trace it from Docker

```
$ docker info --format '{{.OperatingSystem}}'
Docker Desktop
$ docker run --rm alpine uname -r
6.6.87.2-microsoft-standard-WSL2
$ docker context ls
```

Task Manager shows the VM as VmmemWSL. The Hyper-V backend shows a linuxkit kernel.

CONTAINERS ON WINDOWS

# Keep the VM in check

The WSL VM takes memory as it needs it. Set a ceiling.

%UserProfile%\.wslconfig

```
[wsl2]
memory=4GB
processors=2
```

Run wsl --shutdown, then start Docker Desktop again to apply

CONTAINERS ON WINDOWS

# On Oracle Linux: no VM at all

Containers use the server's own kernel directly.

```
$ uname -r
5.15.0-308.179.6.el9uek.x86_64
$ docker run --rm alpine uname -r
5.15.0-308.179.6.el9uek.x86_64
$ docker run --rm oraclelinux:9 cat /etc/oracle-release
```

02

# Linux toolkit

The commands we use around every deployment on Oracle Linux 9



>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Where am I?

```
$ cat /etc/oracle-release
$ hostnamectl              # name, OS, kernel
$ nproc; free -h           # CPUs and memory
$ df -h /                  # free disk space
$ uptime                   # load average
```

>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Files and folders

```
$ ls -lah /opt/blog-app
$ mkdir -p /opt/blog-app/backups
$ cp db/password.txt backups/
$ find . -name "*.py" | wc -l
$ grep -rn "proxy_pass" proxy/
```

>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Reading logs

```
$ tail -f logs/deploy-*.log
$ less compose.yaml
$ journalctl -u docker -f
$ journalctl -b -p err    # errors since boot
```

>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Users and permissions

```
$ id                          # my UID and groups
$ sudo chown -R $USER: /opt/blog-app
$ chmod 600 db/password.txt
$ ls -Z db/                   # SELinux labels
```

>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Network checks

```
$ hostname -I                          # server IP addresses
$ ss -tulpn | grep -E ':(80|8000) '    # who is listening
$ curl -I http://localhost             # does Nginx answer?
$ sudo firewall-cmd --list-all
$ sudo firewall-cmd --add-service=http --permanent
$ sudo firewall-cmd --reload
```

>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Services with systemctl

```
$ systemctl status docker
$ sudo systemctl enable --now docker
$ sudo systemctl restart docker
$ systemctl list-units --failed
```

>_ LINUX TOOLKIT FOR ORACLE LINUX 9

# Packages and long sessions

```
$ sudo dnf install -y git curl tmux
$ dnf list installed | grep -i docker
$ tmux new -s deploy        # a session that survives SSH drops
# detach: Ctrl+B then D
$ tmux attach -t deploy
$ rsync -avz backups/ admin@10.10.1.30:/srv/blog-backups/
```

CHECKPOINT 1 / QUESTION 1

# Why does Docker Desktop on Windows need a virtual machine?

A Windows is too slow

B Linux containers need a Linux kernel

C A licence requires it

D To save disk space

CHECKPOINT 1 / ANSWER 1

# Why does Docker Desktop on Windows need a virtual machine?

A Windows is too slow

B Linux containers need a Linux kernel

C A licence requires it

D To save disk space

Containers share the host kernel, and Windows does not have a Linux one.

CHECKPOINT 1 / QUESTION 2

# Which command lists WSL distributions and their version?

A `wsl --shutdown`

B `systeminfo`

C `wsl -l -v`

D `docker ps`

CHECKPOINT 1 / ANSWER 2

# Which command lists WSL distributions and their version?

A `wsl --shutdown`

B `systeminfo`

C `wsl -l -v`

D `docker ps`

It shows every distro, its state, and whether it runs WSL 1 or 2.

CHECKPOINT 1 / QUESTION 3

# Which command shows the ports listening on a Linux server?

A `df -h`

B `free -h`

C `tail -f`

D `ss -tulpn`

CHECKPOINT 1 / ANSWER 3

# Which command shows the ports listening on a Linux server?

A `df -h`

B `free -h`

C `tail -f`

D `ss -tulpn`

It lists each listening port with the process that owns it.

# 03

# Architecture & Sample



AMS as an architectural case study, and introducing the practice sample

AMS CASE STUDY & PRACTICE PROJECT

# Meet AMS

Batches, students, teachers, centre coordinators and attendance.

- Django 6 on Python 3.13
- MySQL 8 for data
- Tailwind CSS and DaisyUI, built with Node
- Nginx and Gunicorn in production



AMS CASE STUDY & PRACTICE PROJECT

# AMS is live

ams.nielitagt.in, in daily use at NIELIT Agartala

**1.5K+**

users today

**PWA**

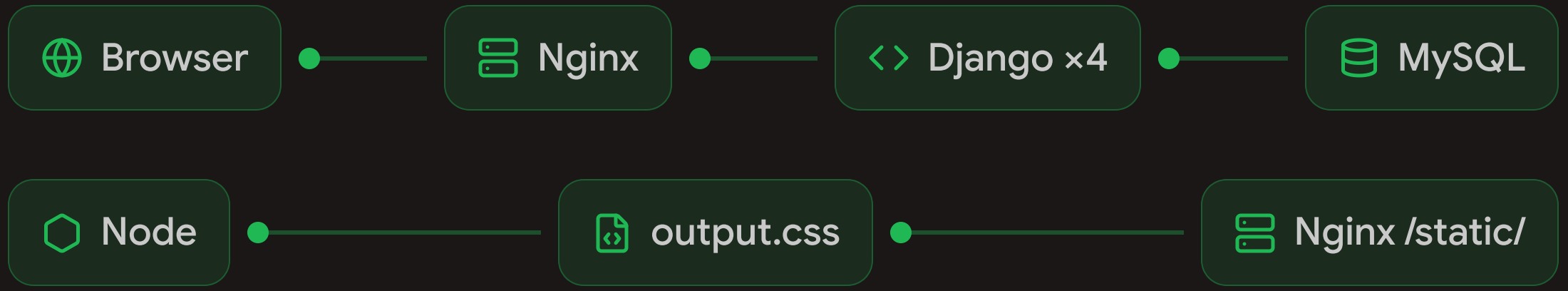
installs like an app

**Docker**

every part in a container

AMS CASE STUDY & PRACTICE PROJECT

# How AMS fits together




Only Nginx faces users. Everything else talks on private networks.

AMS CASE STUDY & PRACTICE PROJECT

# What running AMS taught us

- One compose file describes the whole stack
- Secrets live outside Git
- A deploy script, not manual steps
- SELinux stays on, with labels on every mount
- It must come back by itself after a reboot

AMS CASE STUDY & PRACTICE PROJECT

# Our practice project

Docker's official sample: awesome-compose / nginx-flask-mysql




The same shape as AMS: Nginx, a Python app and a MySQL-family database

AMS CASE STUDY & PRACTICE PROJECT

# The project files

github.com/docker/awesome-compose/nginx-flask-mysql

```
nginx-flask-mysql/
├── compose.yaml            # the whole stack
├── backend/
│   ├── Dockerfile          # Python image with the app inside
│   ├── hello.py            # the Flask application
│   └── requirements.txt
├── proxy/
│   ├── Dockerfile          # Nginx image
│   └── conf                # Nginx configuration
└── db/
    └── password.txt        # the database password
```

AMS CASE STUDY & PRACTICE PROJECT

# compose.yaml: the database

compose.yaml (shortened)

```
services:
  db:
    image: mariadb:10-focal
    restart: always
    healthcheck:
      test: ['CMD-SHELL', 'mysqladmin ping -h 127.0.0.1 --password=... --silent']
      interval: 3s
    secrets: [db-password]
    volumes: ["db-data:/var/lib/mysql"]
    networks: [backnet]
    environment:
      - MYSQL_DATABASE=example
      - MYSQL_ROOT_PASSWORD_FILE=/run/secrets/db-password
```

AMS CASE STUDY & PRACTICE PROJECT

# compose.yaml: backend and proxy

compose.yaml (shortened)

```
  backend:
    build: { context: backend, target: builder }
    restart: always
    secrets: [db-password]
    ports: ["8000:8000"]
    networks: [backnet, frontnet]
    depends_on:
      db: { condition: service_healthy }
  proxy:
    build: proxy
    restart: always
    ports: ["80:80"]
    networks: [frontnet]
secrets:
  db-password: { file: db/password.txt }
```

AMS CASE STUDY & PRACTICE PROJECT

# Three details worth noticing

| | | |
|---|---|---|
| Two networks | → | The proxy can never reach the database |
| Secrets | → | The password is a file, not an environment variable |
| Healthcheck | → | The backend waits until the database is ready |

AMS CASE STUDY & PRACTICE PROJECT

# The backend image

backend/Dockerfile (first stage)

```
FROM python:3.10-alpine AS builder
WORKDIR /code
COPY requirements.txt /code
RUN --mount=type=cache,target=/root/.cache/pip pip3 install -r requirements.txt
COPY . .
ENV FLASK_APP hello.py
ENV FLASK_RUN_PORT 8000
ENV FLASK_RUN_HOST 0.0.0.0
CMD ["flask", "run"]
```

The code is copied into the image: every change needs a rebuild

AMS CASE STUDY & PRACTICE PROJECT

# What the app does

- Reads the password from /run/secrets/db-password
- Connects to the host named db
- On the first request, creates a blog table with four posts
- Returns the post titles as HTML

AMS CASE STUDY & PRACTICE PROJECT

# The proxy

Two lines of Dockerfile: FROM nginx, then copy this file in.

proxy/conf

```
server {
    listen       80;
    server_name  localhost;
    location / {
        proxy_pass   http://backend:8000;
    }
}
```

04

# Preparing the server



Oracle Linux 9 with Docker Engine, and nothing else

PREPARING THE SERVER

# What the server needs

- Oracle Linux 9 and a user with sudo
- Git to fetch the code
- Docker Engine with Compose, or Podman
- Port 80 open in firewalld

No Python, Nginx or database on the host. They all arrive as images.

PREPARING THE SERVER

# Install Docker Engine

```
$ sudo dnf install -y dnf-plugins-core git
$ sudo dnf config-manager --add-repo \
    https://download.docker.com/linux/rhel/docker-ce.repo
$ sudo dnf remove -y podman runc          # they conflict with Docker's packages
$ sudo dnf install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
$ sudo systemctl enable --now docker
```

PREPARING THE SERVER

# Docker without sudo

```
$ sudo usermod -aG docker $USER
$ newgrp docker            # or log out and in
$ docker version
$ docker run --rm hello-world
```

⚠ The docker group is equal to root. Add trusted admins only.

PREPARING THE SERVER

# Or stay with Podman

```
$ sudo dnf install -y container-tools
$ sudo dnf install -y podman-compose    # from EPEL
$ podman run --rm alpine uname -r
$ podman-compose up -d
```

Same images, almost the same commands, and no daemon

PREPARING THE SERVER

# Is the server ready?

```
$ docker compose version
$ docker info | grep -E 'Server Version|Cgroup Version|Storage Driver'
$ getenforce
Enforcing
$ sudo firewall-cmd --add-service=http --permanent
$ sudo firewall-cmd --reload
```

# 05

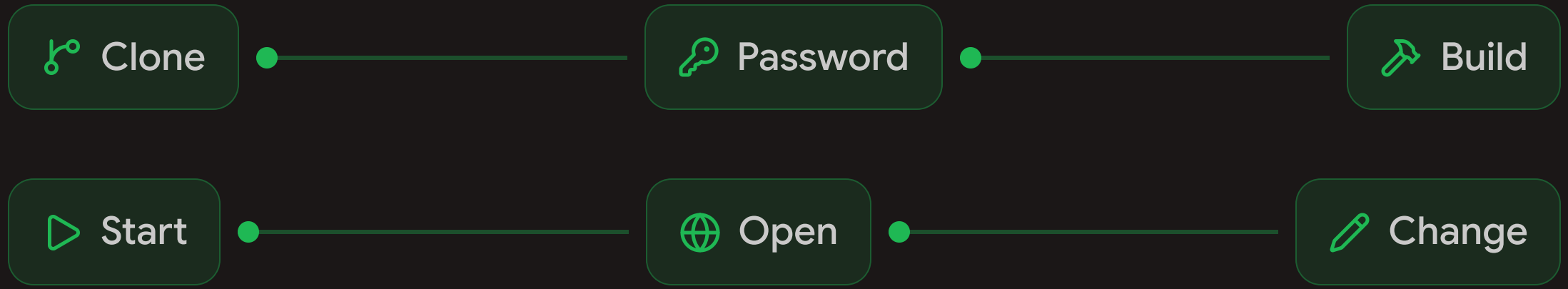
# Deploy it

The sample project, step by step on the live server



DEPLOY THE SAMPLE STEP BY STEP

# The plan



🚀 DEPLOY THE SAMPLE STEP BY STEP

# Step 1: get the code

```
$ git clone --depth 1 https://github.com/docker/awesome-compose
$ sudo cp -r awesome-compose/nginx-flask-mysql /opt/blog-app
$ sudo chown -R $USER: /opt/blog-app
$ cd /opt/blog-app
$ ls
README.md  backend  compose.yaml  db  proxy
```

DEPLOY THE SAMPLE STEP BY STEP

# Step 2: a real password

```
$ cat db/password.txt
db-78n9n
$ printf '%s' "$(openssl rand -hex 12)" > db/password.txt
$ chmod 600 db/password.txt
$ printf '\nWerkzeug==2.0.3\n' >> backend/requirements.txt
```

printf adds no newline. The app reads the file exactly as it is.

🚀 DEPLOY THE SAMPLE STEP BY STEP

# Step 3: check the config

```
$ docker compose config --services
db
backend
proxy
$ docker compose config | less
```

DEPLOY THE SAMPLE STEP BY STEP

# Step 4: build and start

```
$ docker compose up -d --build
$ docker compose ps
NAME                  SERVICE   STATUS
blog-app-backend-1    backend   Up 5 seconds
blog-app-db-1         db        Up 40 seconds (healthy)
blog-app-proxy-1      proxy     Up 4 seconds
```

DEPLOY THE SAMPLE STEP BY STEP

# Step 5: watch it start

```
$ docker compose logs -f db backend
db-1       | mariadbd: ready for connections.
backend-1  |  * Serving Flask app 'hello.py'
backend-1  |  * Running on all addresses.
# Ctrl+C stops following, the containers keep running
```

🚀 DEPLOY THE SAMPLE STEP BY STEP

# Step 6: open it

```
$ curl http://localhost
<div>   Hello  Blog post #1</div><div>   Hello  Blog post #2</div>...
$ hostname -I
```

Browse to http://SERVER_IP

DEPLOY THE SAMPLE STEP BY STEP

# Step 7: look inside the database

```
$ docker compose exec db sh -c \
    'mariadb -uroot -p"$(cat /run/secrets/db-password)" example -e "SELECT * FROM blog"'
+----+--------------+
| id | title        |
+----+--------------+
|  1 | Blog post #1 |
```

DEPLOY THE SAMPLE STEP BY STEP

# Step 8: change the code

```
$ sed -i 's/Hello/Welcome to/' backend/hello.py
$ curl -s http://localhost | head -c 40     # still Hello
$ docker compose up -d --build backend
$ curl -s http://localhost | head -c 40
<div>   Welcome to  Blog post #1</div>
```

DEPLOY THE SAMPLE STEP BY STEP

# Scale the backend?

```
$ docker compose up -d --scale backend=3
Bind for 0.0.0.0:8000 failed: port is already allocated
# remove  ports: ["8000:8000"]  from backend in compose.yaml, then
$ docker compose up -d --scale backend=3
$ docker compose restart proxy
```

DEPLOY THE SAMPLE STEP BY STEP

# Before going live

- A strong password, never committed to Git
- Only the proxy publishes a port
- A production server like Gunicorn, not flask run
- Current, pinned image versions

CHECKPOINT 2 / QUESTION 1

# Where does the sample keep its database password?

A `compose.yaml`

B `backend/hello.py`

C `db/password.txt`

D `proxy/conf`

CHECKPOINT 2 / ANSWER 1

# Where does the sample keep its database password?

A `compose.yaml`

B `backend/hello.py`

C `db/password.txt`

D `proxy/conf`

Compose mounts it into the containers as a secret.

CHECKPOINT 2 / QUESTION 2

# Which command shows the status of every container in the stack?

A `docker compose ps`

B `git status`

C `ls -l`

D `df -h`

CHECKPOINT 2 / ANSWER 2

# Which command shows the status of every container in the stack?

- **A** `docker compose ps`
- **B** `git status`
- **C** `ls -l`
- **D** `df -h`

It also shows whether the database health check passed.

CHECKPOINT 2 / QUESTION 3

# You edited hello.py. What makes the change appear?

A `docker compose restart`

B `docker compose up -d --build`

C `git status`

D Refreshing the browser

CHECKPOINT 2 / ANSWER 3

# You edited hello.py. What makes the change appear?

A `docker compose restart`

B `docker compose up -d --build`

C `git status`

D Refreshing the browser

The code is inside the image, so the image must be rebuilt.

06

# Build without installing



Create and run projects with tools that live only in containers

BUILD WITHOUT INSTALLING ANYTHING

# Your machine needs only Docker

Python, Node, database clients: all of them run as throwaway containers.

- The whole team uses the same versions
- Nothing to uninstall or upgrade later
- Works the same on Windows, Linux and macOS

BUILD WITHOUT INSTALLING ANYTHING

# The pattern: mount, work, remove

```
$ docker run --rm -it \
    -v "$PWD":/work:Z -w /work \
    python:3.13-slim bash
```

| | | |
|---|---|---|
| --rm | → | Delete the container on exit |
| -v "$PWD":/work:Z | → | Share this folder, with an SELinux label |
| -w /work | → | Start inside that folder |

BUILD WITHOUT INSTALLING ANYTHING

# Walkthrough: a new Django project

```
$ mkdir mysite && cd mysite
$ docker run --rm -v "$PWD":/work:Z -w /work python:3.13-slim \
    sh -c "pip install -q django && django-admin startproject mysite ."
$ sudo chown -R $USER: .      # files were created by root
$ ls
manage.py  mysite
```

BUILD WITHOUT INSTALLING ANYTHING

# Walkthrough: run it with Compose

mysite/compose.yaml

```
services:
  web:
    image: python:3.13-slim
    working_dir: /work
    volumes: ["./:/work:Z"]
    ports: ["8000:8000"]
    command: sh -c "pip install -q django && python manage.py runserver 0.0.0.0:8000"
```

```
$ docker compose up -d
$ curl -I http://localhost:8000
```

BUILD WITHOUT INSTALLING ANYTHING

# Frontend tools, same idea

```
$ docker run --rm -v "$PWD":/app:Z -w /app node:24-slim npm init -y
$ docker run --rm -v "$PWD":/app:Z -w /app node:24-slim \
    npm install tailwindcss @tailwindcss/cli
$ docker run --rm -v "$PWD":/app:Z -w /app node:24-slim npx tailwindcss --help
```

AMS builds its CSS exactly like this, with no Node.js on the server

BUILD WITHOUT INSTALLING ANYTHING

# Real use: make a strong password

```
$ docker run --rm python:3.13-slim \
    python -c "import secrets; print(secrets.token_urlsafe(18), end='')" \
    > db/password.txt
$ wc -c db/password.txt
24 db/password.txt
```

end='' means no newline, exactly what our app needs

BUILD WITHOUT INSTALLING ANYTHING

# A database client for free

```
$ docker compose exec db mariadb -uroot -p example
MariaDB [example]> SHOW TABLES;
MariaDB [example]> SELECT COUNT(*) FROM blog;
$ docker compose exec db mariadb-admin -uroot -p status
```

# 07

# Git and automation



Basic Git, a deploy script, webhooks and GitHub Actions

GIT, WEBHOOKS AND GITHUB ACTIONS

# Git in one slide

```
$ git clone URL          # copy a repository
$ git status             # what changed?
$ git add .              # stage the changes
$ git commit -m "feat: welcome message"
$ git push               # send to the server
$ git pull               # get the latest
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# Make the sample your own repo

```
$ cd /opt/blog-app
$ git init -b main
$ printf 'logs/\ndb/password.txt\n' > .gitignore
$ git config --global user.name "Your Name"
$ git add . && git commit -m "feat: blog app from awesome-compose"
$ git remote add origin https://github.com/YOUR_USER/blog-app.git
$ git push -u origin main
```

.gitignore keeps the password out of Git. GitHub asks for a token, not a password.

GIT, WEBHOOKS AND GITHUB ACTIONS

# Branches: never push to main

```
$ git switch -c feat/welcome
$ git commit -am "feat: welcome message"
$ git push -u origin feat/welcome
# then open a Pull Request into main on GitHub
```

AMS follows the same rule: every change arrives as a pull request

GIT, WEBHOOKS AND GITHUB ACTIONS

# Look before you deploy

```
$ git log --oneline --graph -10
$ git fetch
$ git diff HEAD origin/main --stat
$ git tag v1.0 && git push --tags
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# Undo safely

```
$ git restore backend/hello.py          # drop local edits to a file
$ git revert HEAD                       # new commit that undoes the last
$ git reset --hard origin/main          # server: match the remote exactly
```

⚠ reset --hard throws away local changes. Never edit code on the server.

GIT, WEBHOOKS AND GITHUB ACTIONS

# deploy.sh

/opt/blog-app/deploy.sh

```
#!/bin/bash
set -e
cd /opt/blog-app
mkdir -p logs
exec > >(tee -a "logs/deploy-$(date +%F-%H%M%S).log") 2>&1
echo "=== Deploy started at $(date) ==="
git pull --ff-only
docker compose up -d --build
docker image prune -f
echo "Deploy complete."
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# Run it

```
$ chmod +x deploy.sh
$ git add deploy.sh && git commit -m "feat: deploy script"
$ ./deploy.sh
=== Deploy started at Wed Oct 29 11:42:10 IST 2026 ===
Already up to date.
 Container blog-app-db-1  Running
Deploy complete.
$ ls logs/
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# No internet? git bundle

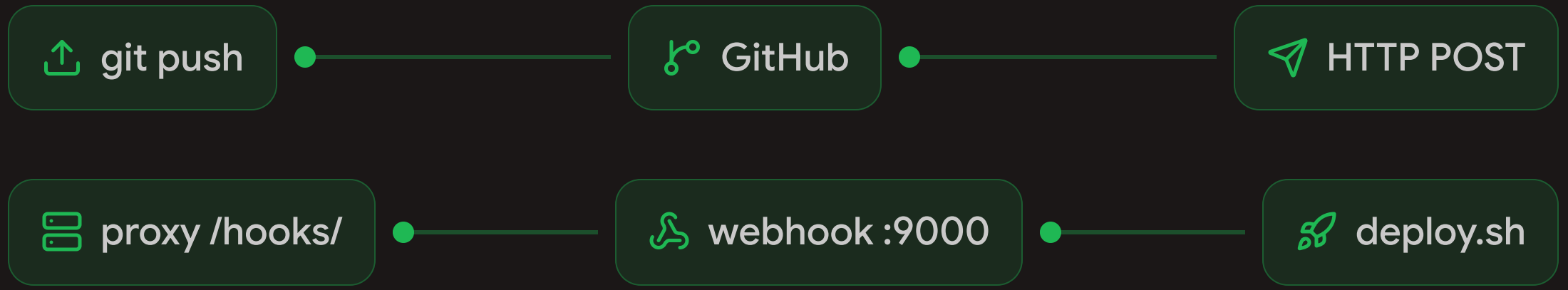
```
# on a machine that can reach GitHub
$ git bundle create blog.bundle main
# copy blog.bundle to the server (scp, USB), then
$ git pull --ff-only /tmp/blog.bundle main
$ docker compose up -d --build
```

A whole repository in one file, for servers that cannot reach GitHub

GIT, WEBHOOKS AND GITHUB ACTIONS

# What is a webhook?

An HTTP request the Git server sends when something happens.



GIT, WEBHOOKS AND GITHUB ACTIONS

# hooks.json

/etc/webhook/hooks.json

```
[{
  "id": "deploy-blog",
  "execute-command": "/opt/blog-app/deploy.sh",
  "command-working-directory": "/opt/blog-app",
  "trigger-rule": { "and": [
    { "match": { "type": "payload-hmac-sha256", "secret": "WEBHOOK_SECRET",
        "parameter": { "source": "header", "name": "X-Hub-Signature-256" } } },
    { "match": { "type": "value", "value": "refs/heads/main",
        "parameter": { "source": "payload", "name": "ref" } } }
  ] }
}]
```

Only signed pushes to main trigger a deploy

# The listener as a service

/etc/systemd/system/webhook.service

```
[Unit]
Description=Webhook listener for blog-app
After=network-online.target docker.service
[Service]
ExecStart=/usr/local/bin/webhook -hooks /etc/webhook/hooks.json -port 9000 -verbose
Restart=always
[Install]
WantedBy=multi-user.target
```

```
$ sudo systemctl enable --now webhook
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# Route /hooks/ through the proxy

proxy/conf (add)

```
location /hooks/ {
    proxy_pass   http://host.docker.internal:9000;
}
```

compose.yaml, under proxy (add)

```
extra_hosts: ["host.docker.internal:host-gateway"]
```

```
$ docker compose up -d --build proxy
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# Test it from the server

```
$ curl -X POST http://localhost/hooks/deploy-blog
Hook rules were not satisfied.
$ BODY='{"ref":"refs/heads/main"}'
$ SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "$SECRET" | cut -d' ' -f2)
$ curl -X POST http://localhost/hooks/deploy-blog \
    -H "X-Hub-Signature-256: sha256=$SIG" -H "Content-Type: application/json" -d "$BODY"
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# Add the webhook on GitHub

- Repository, Settings, Webhooks, Add webhook
- Payload URL: http://SERVER/hooks/deploy-blog
- Content type: application/json, same secret
- Events: just the push event

GitHub must reach that URL from the internet. An internal Gitea can reach internal servers.

GIT, WEBHOOKS AND GITHUB ACTIONS

# GitHub Actions

- A workflow is a YAML file in .github/workflows
- It runs on events: push, pull request, schedule
- Jobs run on runners: GitHub's cloud or your own server



# A workflow for blog-app

.github/workflows/deploy.yml

```yaml
name: Deploy blog-app
on:
  push:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: printf 'ci-only' > db/password.txt
      - run: docker compose build
  deploy:
    needs: build
    runs-on: self-hosted
    steps:
      - run: /opt/blog-app/deploy.sh
```

GIT, WEBHOOKS AND GITHUB ACTIONS

# A self-hosted runner

```
# GitHub: Settings, Actions, Runners, New self-hosted runner
$ mkdir actions-runner && cd actions-runner
# download the runner with the commands GitHub shows, then
$ ./config.sh --url https://github.com/YOUR_USER/blog-app --token TOKEN
$ sudo ./svc.sh install && sudo ./svc.sh start
```

The runner connects out to GitHub: no inbound port needed

GIT, WEBHOOKS AND GITHUB ACTIONS

# Webhook or Actions?

## Webhook

Tiny, runs on your server

Git server must reach your URL

You write the checks

## Actions

Build and test before deploying

Logs and history on GitHub

Runner needs no inbound port

GIT, WEBHOOKS AND GITHUB ACTIONS

# Try it end to end

- Edit backend/hello.py on GitHub, on a new branch
- Open a pull request and merge it into main
- GitHub calls the webhook, or the workflow runs
- Refresh the browser: the new text is live

CHECKPOINT 3 / QUESTION 1

# Which command creates a new branch and switches to it?

A `git push`

B `git clone`

C `git log`

D `git switch -c fix/x`

CHECKPOINT 3 / ANSWER 1

# Which command creates a new branch and switches to it?

A `git push`

B `git clone`

C `git log`

D `git switch -c fix/x`

Then commit, push the branch and open a pull request.

CHECKPOINT 3 / QUESTION 2

# What sends an HTTP request to your server when code is pushed?

A A webhook

B A volume

C A Dockerfile

D tmux

CHECKPOINT 3 / ANSWER 2

# What sends an HTTP request to your server when code is pushed?

- **A** A webhook
- **B** A volume
- **C** A Dockerfile
- **D** tmux

The listener checks the signature, then runs deploy.sh.

CHECKPOINT 3 / QUESTION 3

# Where do GitHub Actions workflow files live?

A `/etc/systemd/system`

B `proxy/conf`

C `.github/workflows`

D `logs/`

CHECKPOINT 3 / ANSWER 3

# Where do GitHub Actions workflow files live?

A `/etc/systemd/system`

B `proxy/conf`

C `.github/workflows`

D `logs/`

One YAML file per workflow, committed with the code.

08

# Troubleshooting



Find the problem, remove things safely, survive a reboot

TROUBLESHOOTING AND KEEPING IT RUNNING

# Work through it in order




Is it running? What did it say? What does it see? What was it given?

TROUBLESHOOTING AND KEEPING IT RUNNING

# Is it running? What did it say?

```
$ docker compose ps -a
$ docker compose logs --tail 50 backend
$ docker compose logs -f proxy db
$ docker compose top
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Get inside with exec

```
$ docker compose exec backend sh                # Alpine has sh, not bash
$ docker compose exec backend ls -l /run/secrets
$ docker compose exec -u root proxy sh          # as root
$ docker compose exec -T db sh -c \
    'mariadb -uroot -p"$(cat /run/secrets/db-password)" -e "SHOW DATABASES"'
```

-T turns off the terminal: needed in scripts, cron jobs and webhooks

TROUBLESHOOTING AND KEEPING IT RUNNING

# What was it given?

```
$ docker compose config                    # the final config
$ docker compose exec backend env | grep FLASK
$ docker inspect blog-app-db-1 --format '{{.State.Health.Status}}'
$ docker stats --no-stream
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Common problems

| | | |
|---|---|---|
| Port 80 in use | → | Stop the host's httpd, or map 8080:80 |
| Access denied | → | Password changed after the first start |
| Secret not found | → | Create db/password.txt on the server |
| 502 Bad Gateway | → | Backend down: docker compose logs backend |
| Change not live | → | The code is in the image: up -d --build |

TROUBLESHOOTING AND KEEPING IT RUNNING

# A gotcha: the password file

- The app reads the file exactly, newline included
- MariaDB sets the password only when the volume is empty
- Editing password.txt later breaks the login

Change it inside MariaDB with ALTER USER, then update the file

TROUBLESHOOTING AND KEEPING IT RUNNING

# Removing things

```
$ docker compose stop backend          # stop, keep the containers
$ docker compose rm -sf proxy          # stop and remove one service
$ docker compose down                  # remove containers and networks
$ docker compose down -v               # ...and the volumes: database gone
$ docker compose down --rmi local      # ...and the images we built
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Cleaning up disk

```
$ docker system df                          # what uses the space
$ docker image prune -a                     # unused images
$ docker builder prune                      # build cache
$ docker volume ls
$ docker volume rm blog-app_db-data         # every record, gone
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Why removing hurts

- down -v deletes the database, with no confirmation
- The next start makes a new, empty database
- Rebuilding images means minutes of downtime
- After down, nothing comes back on a reboot

TROUBLESHOOTING AND KEEPING IT RUNNING

# Back up before you remove

```
$ docker compose exec -T db sh -c \
    'mariadb-dump -uroot -p"$(cat /run/secrets/db-password)" example' > backup.sql
$ cp db/password.txt backups/
# restore
$ docker compose exec -T db sh -c \
    'mariadb -uroot -p"$(cat /run/secrets/db-password)" example' < backup.sql
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Who brings it back after a reboot?

| | | |
|---|---|---|
| restart: always | → | Yes, if the docker service is enabled |
| No restart policy | → | No, it stays stopped |
| Podman | → | Only if podman-restart is enabled |
| After down | → | No, the containers are gone |

TROUBLESHOOTING AND KEEPING IT RUNNING

# Fix 1: restart policies

compose.yaml

```
backend:
  restart: always       # or unless-stopped
proxy:
  restart: always
```

```
$ sudo systemctl enable docker          # Podman: enable podman-restart
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Fix 2: a service for the stack

/etc/systemd/system/blog-app.service

```
[Unit]
Description=blog-app containers
Requires=docker.service
After=docker.service network-online.target
[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/blog-app
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down
[Install]
WantedBy=multi-user.target
```

TROUBLESHOOTING AND KEEPING IT RUNNING

# Use the service

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now blog-app
$ systemctl status blog-app
$ sudo reboot      # then: docker compose ps
```

On Podman, Quadlet files in /etc/containers/systemd do the same job

09

# Subdomains

One address for every build, named by its image ID



SUBDOMAINS WITH IMAGE IDS

# One server, many versions

Every build gets its own address, like 3f2a1c9b8d7e.apps.example.in

- Test a change before it replaces production
- Keep the previous version one URL away
- The image ID names the container and the subdomain

SUBDOMAINS WITH IMAGE IDS

# Step 1: wildcard DNS

DNS zone for example.in

```
*.apps.example.in.    A    10.10.1.20
```

Every name under apps.example.in reaches the server. Without DNS, test with curl -H "Host: ...".

SUBDOMAINS WITH IMAGE IDS

# Step 2: build two versions

```
$ printf 'FROM nginx:alpine\nCOPY index.html /usr/share/nginx/html/\n' > Dockerfile
$ echo "Version 1" > index.html && docker build -t site:v1 .
$ echo "Version 2" > index.html && docker build -t site:v2 .
$ docker images site
REPOSITORY   TAG   IMAGE ID       CREATED
site         v2    7c41d0e2a9b3   5 seconds ago
site         v1    3f2a1c9b8d7e   20 seconds ago
```

SUBDOMAINS WITH IMAGE IDS

# Step 3: a container per image ID

```
$ docker network create apps
$ for t in v1 v2; do
    id=$(docker images -q site:$t)
    docker run -d --name site-$id --network apps site:$t
  done
$ docker ps --format '{{.Names}}'
site-7c41d0e2a9b3
site-3f2a1c9b8d7e
```

SUBDOMAINS WITH IMAGE IDS

# Step 4: route by subdomain

router.conf

```
server {
  listen 80;
  server_name ~^(?<id>[a-f0-9]{12})\.apps\.example\.in$;
  resolver 127.0.0.11 valid=10s;      # Docker's internal DNS
  location / {
    proxy_pass http://site-$id;
    proxy_set_header Host $host;
  }
}
```

SUBDOMAINS WITH IMAGE IDS

# Step 5: start the router and test

```
$ docker run -d --name router --network apps -p 8081:80 \
    -v "$PWD"/router.conf:/etc/nginx/conf.d/default.conf:Z nginx:alpine
$ curl -H "Host: 3f2a1c9b8d7e.apps.example.in" http://localhost:8081
Version 1
$ curl -H "Host: 7c41d0e2a9b3.apps.example.in" http://localhost:8081
Version 2
```

SUBDOMAINS WITH IMAGE IDS

# For Django: allow the subdomains

.env

```
ALLOWED_HOSTS=.apps.example.in
CSRF_TRUSTED_ORIGINS=https://*.apps.example.in
```

A leading dot in ALLOWED_HOSTS matches every subdomain

SUBDOMAINS WITH IMAGE IDS

# Roll back by ID

```
$ docker images --format '{{.ID}}  {{.Tag}}  {{.CreatedSince}}' site
$ docker rm -f site-7c41d0e2a9b3       # drop the bad version
# 3f2a1c9b8d7e.apps.example.in still serves v1
$ docker image prune                   # tidy up later
```

CHECKPOINT 4 / QUESTION 1

# Which command deletes the database volume?

A `docker compose ps`

B `docker compose down -v`

C `docker compose logs`

D `docker compose restart`

CHECKPOINT 4 / ANSWER 1

# Which command deletes the database volume?

A `docker compose ps`

B `docker compose down -v`

C `docker compose logs`

D `docker compose restart`

-v removes named volumes. Take a backup first.

CHECKPOINT 4 / QUESTION 2

# What brings the app back after the server reboots?

A `git pull`

B `docker compose logs`

C A systemd service or restart policy

D A bigger disk

CHECKPOINT 4 / ANSWER 2

# What brings the app back after the server reboots?

A `git pull`

B `docker compose logs`

C A systemd service or restart policy

D A bigger disk

Restart policies plus an enabled engine, or blog-app.service.

CHECKPOINT 4 / QUESTION 3

# What does a wildcard record like *.apps.example.in do?

A Sends every subdomain to one server

B Blocks all subdomains

C Deletes old images

D Encrypts the traffic

CHECKPOINT 4 / ANSWER 3

# What does a wildcard record like *.apps.example.in do?

- **A** Sends every subdomain to one server
- **B** Blocks all subdomains
- **C** Deletes old images
- **D** Encrypts the traffic

Nginx then decides which container answers each name.

WRAP-UP

# Key takeaways

- On Windows, Linux containers run in WSL 2 or Hyper-V
- On Oracle Linux, they use the server's own kernel
- One compose file runs the proxy, the app and the database
- Push to main, then a webhook or Actions runs deploy.sh
- Back up before down -v, and use a service for reboots
- Image IDs plus subdomains give previews and rollback

THANK YOU

# Questions and Discussion

Deploying a Live Project with Docker

